

ЛАБОРАТОРНАЯ РАБОТА №18 (6+6+8 БАЛЛОВ)

Вариант	Задача 1	Задача 2	Задача 3
1	TreeFun1	TreeFun3	TreeFun11
2	TreeFun2	TreeFun4	TreeFun12
3	TreeFun1	TreeFun5	TreeFun13
4	TreeFun2	TreeFun6	TreeFun9
5	TreeFun1	TreeFun7	TreeFun10
6	TreeFun2	TreeFun8	TreeFun11
7	TreeFun1	TreeFun3	TreeFun12
8	TreeFun2	TreeFun4	TreeFun13
9	TreeFun1	TreeFun5	TreeFun9
10	TreeFun2	TreeFun6	TreeFun10
11	TreeFun1	TreeFun7	TreeFun11
12	TreeFun2	TreeFun8	TreeFun12
13	TreeFun1	TreeFun3	TreeFun13
14	TreeFun2	TreeFun4	TreeFun9
15	TreeFun1	TreeFun5	TreeFun10

БИНАРНЫЕ ДЕРЕВЬЯ И СПИСКИ

TreeFun1. Преобразовать бинарное дерево поиска в двусвязный список без использования дополнительной памяти (создания новых объектов). При преобразовании поля left и right узлов бинарного дерева рассматриваются эквивалентными полям prev и next узлов двусвязного списка. Вывести исходное дерево и получившийся список. Элементы в результирующем списке должны сохранить свою упорядоченность.

TreeFun2. Преобразовать двусвязный список в бинарное дерево поиска без использования дополнительной памяти (создания новых объектов). Корнем дерева должен стать элемент списка, находящийся в его середине, а само дерево должно иметь наименьшую возможную высоту. При преобразовании поля left и right узлов бинарного дерева рассматриваются эквивалентными полям prev и next узлов двусвязного списка. Вывести исходный список и получившееся дерево.

ОБХОД БИНАРНОГО ДЕРЕВА

TreeFun3. Реализовать для бинарного дерева интерфейс итератора, который будет возвращать значения элементов, находящихся в узлах дерева, в порядке "лево-корень-право". Преобразовывать дерево в список или иную структуру данных нельзя, рекурсию использовать запрещается.

TreeFun4. Реализовать для бинарного дерева интерфейс итератора, который будет возвращать значения элементов, находящихся в узлах дерева, в порядке "право-корень-лево". Преобразовывать дерево в список или иную структуру данных нельзя, рекурсию использовать запрещается.

TreeFun5. Реализовать для бинарного дерева интерфейс итератора, который будет возвращать значения элементов, находящихся в узлах дерева, в порядке "лево-право-корень". Преобразовывать дерево в список или иную структуру данных нельзя, рекурсию использовать запрещается.

TreeFun6. Реализовать для бинарного дерева интерфейс итератора, который будет возвращать значения элементов, находящихся в узлах дерева, в порядке "корень-право-лево". Преобразовывать дерево в список или иную структуру данных нельзя, рекурсию использовать запрещается.

TreeFun7. Реализовать обход кромки кроны бинарного дерева по часовой стрелке. Обход начинается в корне дерева.

TreeFun8. Реализовать обход кромки кроны бинарного дерева против часовой стрелки. Обход начинается в корне дерева.

ИССЛЕДОВАНИЕ БИНАРНЫХ ДЕРЕВЬЕВ

TreeFun9. Ученые изучают поведение птиц, вьющих гнезда на бинарном дереве, и хотят разместить в его узлах камеры. Каждая камера может обозревать узел, в котором она расположена, а также непосредственного предка и непосредственных потомков этого узла. По заданному бинарному дереву требуется определить, какое минимальное количество камер потребуется ученым для того, чтобы полностью покрыть наблюдением все узлы дерева.

TreeFun10. Ученые изучают поведение птиц, вьющих гнезда на бинарном дереве, и хотят разместить в его узлах камеры. Каждая камера может обозревать узел, в котором она расположена, а также непосредственного предка и непосредственных потомков этого узла. По заданному бинарному дереву требуется определить, какое количество узлов будет покрыто более чем одной камерой при самом оптимальном их размещении.

TreeFun11. В одном из узлов бинарного дерева начинается пожар. Пожар распространяется только по ветвям во все стороны (и на потомков, и на родителей) с одинаковой скоростью. Для заданного бинарного дерева и выбранного узла, с которого начинается пожар, требуется вывести все узлы дерева в том порядке, в котором они будут сгорать.

TreeFun12. Дано бинарное дерево с целыми числами в узлах и некоторое целое число S . Нужно найти в дереве все возможные пути, на которых сумма значений узлов будет равна S .

TreeFun13. Дано бинарное дерево произвольного вида, не являющееся бинарным деревом поиска. Требуется найти и удалить все дублирующиеся поддеревья, начиная с самых крупных.